

RESEARCH ARTICLE

Devolving IEEE 802.1X authentication capability to data plane in software-defined networking (SDN) architecture

Kamal Benzekki*, Abdeslam El Fergougui and Abdelbaki El Belrhiti El Alaoui

Laboratory of Computer Networks and Systems, Department of Mathematics and Computer Sciences, Faculty of Science, Moulay Ismail University, Meknes, Morocco

ABSTRACT

Software-defined networking (SDN) is a relatively new approach in network management that proposes to separate the network control (Control plane) and the forwarding process (Data plane) to optimize the network infrastructure and improve network performance, controllability, manageability and flexibility. However, like every technology, SDN has brought its own new challenges in terms of security and scalability which are very important aspects that should be considered to design and build a resilient architecture in order to meet carrier grade network requirements. In this paper, we propose a secure SDN architecture with IEEE 802.1X port-based authentication where we also consider the controller's scalability issue by devolving the access control capability to the data plane. In this way, we reduce the high demand and the workload on the SDN controller. Our proposed model presents a novel SDN network architecture and logical network segmentation which provides an optimal and secure network access with low latency. We have implemented and tested our architecture to show its performance (authentication delays). Copyright © 2016 John Wiley & Sons, Ltd.

KEYWORDS

software-defined networking; IEEE 802.1X port-based authentication; network access; scalability

*Correspondence

Kamal Benzekki, Faculty of Science, Moulay Ismail University, Meknes, Morocco.

E-mail: benzekki@gmail.com

1. INTRODUCTION

The advent of Software-defined networking (SDN) [1] paradigm has brought new features and offered more flexibility, and eliminated the network complexity of the classical networks. SDN allows easy manageability and controllability on the basis of standards-based software abstraction of the network infrastructure. This abstraction is achieved through the separation of the control plane (control logic) from the data plane (forwarding plane). In this network model, the routers and the switches are only limited to forwarding traffic based on the installed flow rules. The control logic is centralized in a controller which represents the network operating system of an SDN network and plays important roles that consist of managing flow control and configuring network elements of the data plane using protocols such as OpenFlow [2]. OpenFlow is an open standard communication interface which is the most widely adopted in SDN-based architectures. This standard allows defining communications between the

control and the data planes, and depicting instructions and parameters to the forwarding devices such as packets' destination port, Virtual Local Area Network (VLAN) tags and many other characteristics that are established as flow tables. Each router or switch in the forwarding plane, maintains a set of flow tables that enable taking decision regarding the network flows. The OpenFlow communications between the controller and the underlying infrastructure can either be encrypted using Transport Layer Security (TLS) or unprotected with Transmission Control Protocol. In most controllers' implementations TLS encryption is considered as an optional feature.

While SDN focuses on providing enhanced networking features, it has implicitly brought some new security challenges and concerns. Some of them are inherited from the legacy network; others are inherent and specific to SDN protocols and architecture. Security measures are not SDN built-in features, and thus, this may pose serious issues which might lead to the emergence of network vulnerabilities and threats. Security aspects including authentication,

confidentially, integrity, availability and non-repudiation should be considered in order to design and build a dependable and resilient SDN that meets the requirements of today's network communications.

OpenFlow-based SDN architecture can be prone to several attack vectors [3,4] which we can classify into three types: fingerprinting/scanning, access and denial of service. Fingerprinting and scanning are passive attacks that aim at analyzing the traffic, collecting and analyzing information in order to locate vulnerable SDN components, identify active services, open ports and picture the overall network topology from the IP addresses so as to facilitate the planning for further attacks. Usually, this type of attacks is coupled with one or the two other types of attacks in order to lead partially or fully successful penetrations or alterations. Attackers could target the network elements by exploiting software or network vulnerabilities to gain access to network hosts for the purpose of spying and/or tampering communications (Man in the middle) or exhausting resources which is known as denial-of-service attacks.

As the switches are the main local entrance door to SDN, we suggest in our work to secure this door in order to avoid unauthorized access and block any intruder who might intercept the network traffic as a part of fingerprinting or scanning. Besides, we make denial of service attacks hard to achieve as long as the attacker will not be allowed to access the network. Our proposal is a devolvement/delegation of IEEE 802.1X port-based authentication [5] capability from the SDN controller to an IEEE 802.1X legacy switch.

IEEE 802.1X offers secure and flexible authentication mechanisms which facilitate dynamic VLAN assignments. Dynamic VLAN offers mobility, enhanced security and performance through the segmentation of the network into multiple broadcast domains. Hosts belonging to the same VLAN can only communicate between them and are not allowed to send or receive traffic unless explicit rules are defined for this purpose. Also, one more advantage of dynamic VLAN over the static VLAN is that it supports instant mobility of users. For instance, when a given user changes his location and plug his computer into another port, the dynamic VLAN will automatically define and set his membership to the appropriate VLAN on the basis of VLAN name or ID.

As SDN decouples the control logic from the forwarding devices and delegates the control to the brain of the network (controller), network visibility and controllability become more simple to manage but such design can be a double edged sword. In fact, being the center of network, make the controller a single point of failure for the whole network infrastructure, and seriously, its security and scalability [6,7] become questionable as follows:

- (1) How to secure such a critical component serving and interacting in real time with the management, application and data planes?

- (2) How to ensure handling a growing traffic under overload?
- (3) Is the controller capable of self-defending and countering the network threats proactively?

In this paper, we will provide a dependable hybrid architecture where the scalability is a design consideration in the access control mechanism.

The remainder of this paper is organized as follows. Section 2 discusses the existing works which deals with SDN authentication and access control. Section 3 presents our adopted hybrid architecture which adopts legacy switch as an authenticator instead of the SDN controller. Theoretical security analysis and performance studies are presented in subsection 3.2. A network evaluation of our architecture is given in subsection 3.3, and finally, conclusion is provided in Section 4.

2. RELATED WORKS

Software-defined networking offers a new network design enabling a wide range of network applications and services. However, security concerns are emerging, and new challenges to face new treats have appeared. Security is not yet a built-in feature in the SDN architecture, and thus, multiple research papers have been proposed to tackle SDN security issues. In this section, we will focus on similar works to ours for securing network access.

In [8] Matias *et al.*, proposed Flow-based Network Access Control (FlowNAC) which is a Network Access Control solution to authorize legitimate access into the network. FlowNAC is a modified version of IEEE 802.1X standard and has the advantage to authorize access on the basis of flow nature. The network access can be guaranteed for multiple services where each service is identified with specific flows. Being based on SDN, FlowNAC uses the same concept in decoupling the role of the Policy Enforcement Point (PEP). In FlowNAC-based SDN implementation, the PEP consists of three elements: SDN datapath, Authenticator Network Function and SDN controller. The SDN data-path is a flow-based forwarding element (i.e. OpenFlow data-path) which aims to identify Extensible Authentication Protocol over LAN (EAPoL) traffic and service traffic. The Authenticator Network Function analyses, encapsulates and forwards the incoming EAPoL frames to the authentication server (AS) (e.g. RADIUS). Finally, the SDN controller enables configuring the SDN data-path by defining flow entries to establish rules for EAPoL frames and service traffic.

The main purpose of FlowNAC is to perform fine-grained flow-based network access control. However, the authentication delays can be longer than those of IEEE 802.1X port-based authentication.

In [9] Mattos *et al.*, proposed Authentication Flow (AuthFlow) which is an authentication and access control mechanism. AuthFlow is based on IEEE 802.1X and uses credentials in order to verify a host identity and

determine the privileges. This mechanism runs over an architecture consisting of a POX controller, an authenticator and an AS. The authenticator is modified version of hostapd which handles IEEE 802.1X packets forwarded by a POX application. hostapd uses a secure channel Secure Socket Layer to confirm authentication to the POX application. AuthFlow mechanism uses a RADIUS server to authenticate hosts against a Lightweight Directory Access Protocol (LDAP) database based on the received credentials from the supplicant over Extensible Authentication Protocol (EAP) channel.

In their implementation, the authors used MSCHAP v2 in the EAP authentication method, which uses username and password combination. In the authentication process the POX application allows the hosts to have access into the network if the provided credentials are correct. Otherwise, if the credentials do not match any entry in the LDAP database, then, the authenticator informs the POX controller which denies and prevents the supplicant from accessing to the network resources. Skip line to introduce [10], Yamasaki *et al.*, presented a flexible access management system based on OpenFlow. The access management mechanism is based on virtual network group ID (GID) instead of VLANs. GIDs are managed in the access management function (AMF) which is an additional function into the OpenFlow controller. AMF consists of databases of virtual group GIDs, a source and a destination address check function as well as an access control lists check function. Additionally, it integrates GID DBs containing GIDs, port, MAC and Internet Protocol (IP) addresses. When the user wants to authenticate, he contacts the RADIUS server to check his identity on the basis of the MAC address, then, the RADIUS sends a report to OpenFlow controller in case of successful authentication. Afterwards, the user host is permitted and can send a Dynamic Host Configuration Protocol (DHCP) request as soon as the Access Point receives Access-Accept packet from the RADIUS server. The DHCP request is first forwarded to OF controller where AMF takes the appropriate decision regarding this request, and then, the node can be assigned an IP address. Further communications initiated by the host to network resources are checked by AMF. The proposed system has been introduced to eduroam. However, the large databases of this system and their complexity requires high maintenance processes. Moreover, the system does not support anonymous user authentication mode which is provided by some EAP methods such as Protected Extensible Authentication Protocol (PEAP) and EAP-TTLS.

In [11], Kinoshita *et al.*, proposed an Access Control System for Wireless LAN roaming. The authors pinpointed the shortcomings of conventional eduroam system that has been proposed by Trans-European Research and Education Network Association for Wireless LAN roaming between research and educational institutions. Eduroam relies on IEEE 802.1X and a hierarchical tree of RADIUS proxies. However, eduroam suffers from some limitations specific

to EAP methods as well as the maintenance cost of the databases. So, the authors suggest new role information for service providers policy.

3. 802.1X PORT-BASED AUTHENTICATION ARCHITECTURE

3.1. Network architecture

The goal of IEEE 802.1X is to restrict unauthorized hosts from accessing the network with the help of three elements as follows:

- (1) *Supplicant*: Supplicant/client is the device attempting to gain access to wired or wireless network by sending credentials from local software to the authenticator. The exchanged data between the supplicant and the authenticator are held on EAP and encapsulated in EAPOL frames.
- (2) *Authenticator*: The authenticator is typically a switch or a wireless access point, which is located between the supplicant and the Authentication Server (AS). The main role of this network component is to forward authentication information from the supplicant to the AS. Communications occurring in the AS side are generally based on RADIUS [12] protocol.
- (3) *Authentication Server*: Typically the AS is a RADIUS server that authenticates one or multiple clients based on the provided credentials or certificates. The credentials are checked against a local database (e.g. LDAP) to verify the identities.

Figure 1 presents our proposed secure 802.1X Port-based authentication architecture for SDN, which consists of Hosts (supplicants), a legacy switch with IEEE 802.1X support, OpenFlow Switches, DHCP server, RADIUS server, and a SDN controller. OpenFlow network might consist of one or more OF-Switches. In the other part, we have the conventional IEEE 802.1X with OpenFlow-based SDN where the DHCP and RADIUS servers are placed near the SDN controller for better performance and latency reduction purposes.

The behavior of the authentication process of the conventional SDN architecture illustrated in Figure 1(a), is described as follows:

- (1) OpenFlow Switch 1 is used to capture or inject 802.1X/EAPOL messages from the supplicants (hosts) depending on the installed rules. If the OF-switch are in proactive mode the switches applies straightforwardly the predefined rules. Otherwise, in case of reactive mode, the OF-Switch must contact the controller in order to set up new rules.
- (2) Application running on the SDN controller plays the role of the IEEE 802.1X authenticator and forwards the requests to the RADIUS server for challenges and user validation. The authentication results indi-

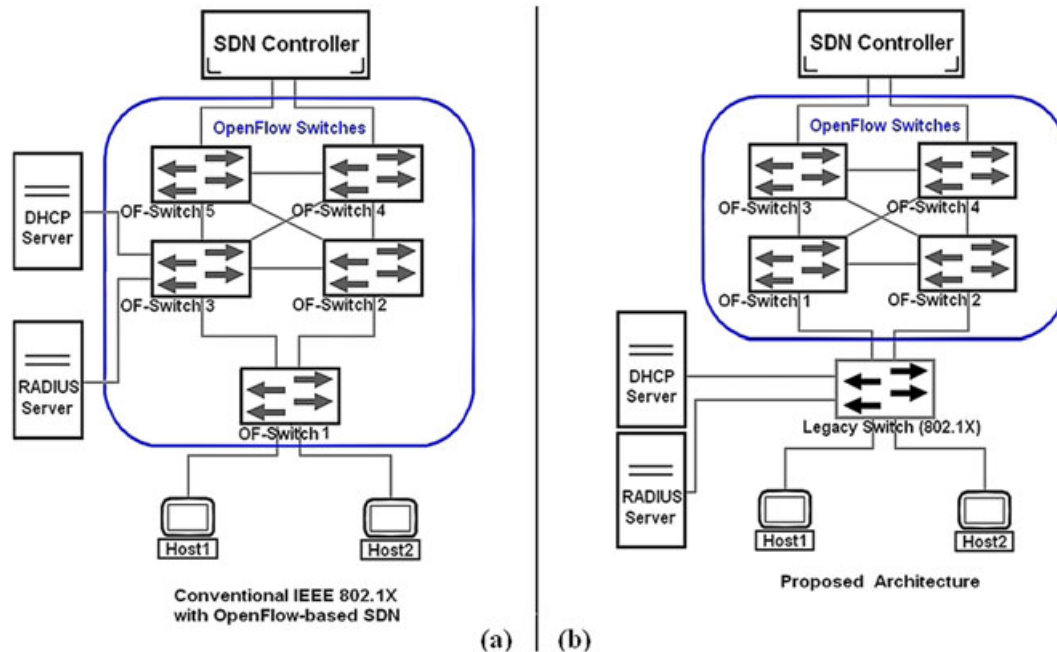


Figure 1. The conventional IEEE 802.1X with OpenFlow-based software-defined networking (SDN) versus the proposed architecture.

cate to the controller which new specific rules to be deployed on the OF-Switches in order to define the user membership.

For learning the available authenticators in the SDN, the IEEE 802.1X supplicant uses a multicast address 01:80:C2:00:00:03 to sends an EAPOL-Start frame. Once an Authenticator is found in the network, it sends an EAP Request/Identity to the supplicant whereby it learns the MAC address of the Authenticator (Controller). The OF-switches identifies IEEE 802.1X flow by EtherType (0x888E).

In our architecture, the OpenFlow controller is not invoked in the authentication process, which eliminates the additional workload (Figure 1(b)). The supplicant is authenticated through the legacy switch which is the main entrance door to SDN. The authentication flow (Either malicious or benign) will never reach the SDN unless the network access is guaranteed after a successful authentication.

All possible IEEE 802.1X authentication scenarios are depicted in the generic flowchart (Figure 2).

3.2. Scalability and performance: theoretical analysis

3.2.1. Scalability.

One of the major issues of SDN technology is the scalability. In our context, scalability defines the ability of an SDN controller to handle and process a growing workload. Scalability aims at improving the capability of the con-

troller by adopting some techniques which can be divided into the following three types:

- (1) Controller capacity improvement (i.e. High Processing [13])
- (2) Distributed event-based control plane (i.e. Clustering [14])
- (3) Delegating some management functions to the data plane (i.e. Devolving [15])

As SDN design enables pushing all the control functionality to a centralized controller with a complete network-wide view, developing network control applications and enforcing policies become much easier with these properties. However, controllers might potentially become a bottleneck for the network operations.

As the size of network grows, more events and requests (e.g. IEEE 802.1X authentications and periodic re-authentications) are sent to the controller, and at some point, handling all the incoming IEEE 802.1X authentication requests by the controller during overloads becomes a serious challenge. This is a common problem that exists in any system and does not only apply to SDN design. By decoupling the control and data planes and entrusting the controller for the control of network traffic over a large network, the growth of network traffic may not scale with the capabilities of the controller in terms of performance, and thus, engendering scalability and elasticity issues [16,17].

Multiple works have been proposed to alleviate the workload on one single controller by distributing and sharing tasks between multiple controllers. These are only workarounds to solve specific concerns of SDNs overhead

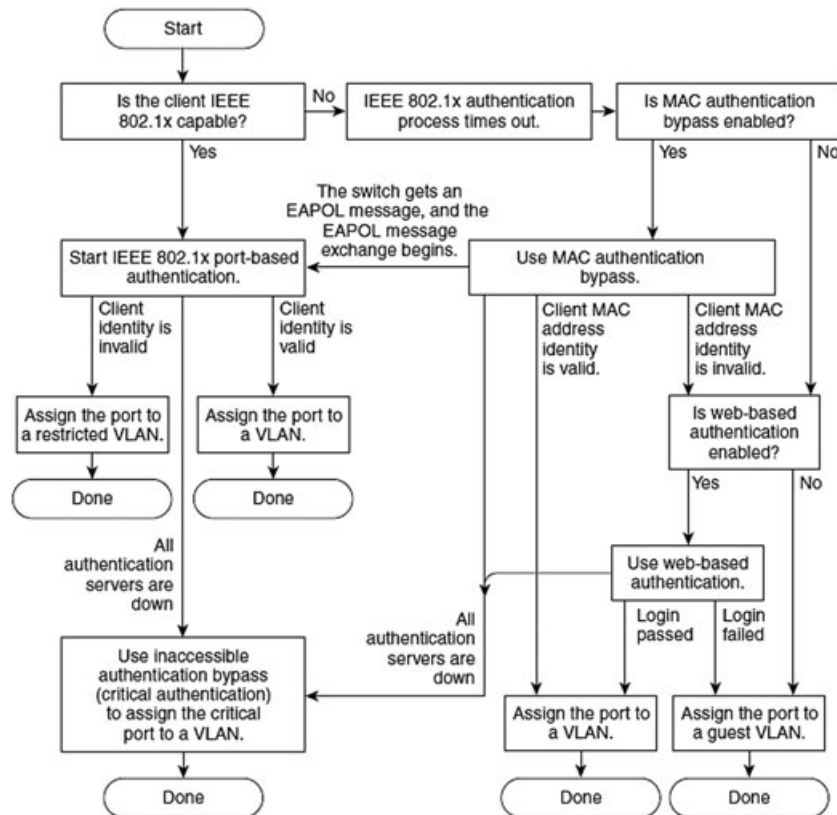


Figure 2. Global IEEE 802.1X authentication flowchart.

on the control plane as well as the high latency concern. These proposed scalability solutions include DIFANE [15], DevoFlow [18], Software-Defined Counters (SDCs) [19], Onix [20], Kandoo [21], HyperFlow [14], Maestro [22] and NOX-MT [23], which mainly aim at reducing the overhead of the control plane by delegating some work to the forwarding devices or trying to increase the capacity of the controllers by implementing high performance computing techniques such as buffering, pipelining, multi-threading and parallelism.

3.2.2. Performance.

Software-defined networking offers flexible and programmable functionalities as well as a decrease in network systems and equipment costs. However, the offered capabilities and features of SDN infrastructure influence the whole network performance (e.g. latency, jitter, throughput and error rate). In fact, SDN may pay performance penalties because of workload of the network traffic that the OpenFlow-based SDN [24] systems handle. The control plane is the main bottleneck of the SDN architecture where performance is a critical issue especially in large scale network implementations because heavy request load on a SDN controller might result in longer delays. For instance, while processing a growing load and receiving multiple IEEE 802.1X messages, the controller may experience

long response delay time and cause authentication timeouts or very slow authentication process for end users.

In our architecture, we delegated/devolved the authentication functionalities to the IEEE 802.1X legacy switch in order to avoid long authentication delays because of scalability and network performance issues. Besides, our proposal enables reducing the latency and the bandwidth usage, as the network interactions are occurring in smaller network segments which include the legacy switch and RADIUS/DHCP servers. In SDN architecture, the authentication flow involves other switches, the controller and additional network paths to forward IEEE 802.1X messages which panelize the bandwidth and increase the response time.

Overall, the proposed system has relatively average computational complexity as the execution time is directly proportional to the number of the authentication requests in a linearithmic mode (Figure 7). The computational complexity of our system can be represented in a linearithmic time of $O(n \log n)$. In our implementation, we adopted PEAP-EAP-MSCHAPv2 for authentication which is designed for average computational complexity if compared with other mechanisms that use certificates and require Public Key Infrastructure to perform advanced cryptographic operations and computations. In fact, certificates require a lot more complex coding process and powerful stations.

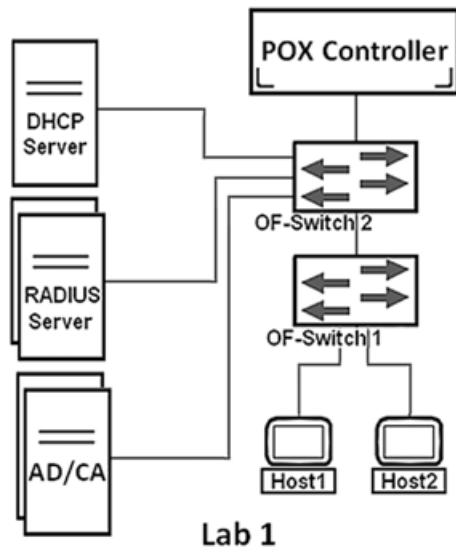


Figure 3. Lab 1

Futhermore, EAP is quite complex and the communication overhead are expensive. Every full EAP-based 802.1x authentication leads to 1000-ms latency. The authentication process may experience a little bit longer delay in the multihop scenario where the authentication messages are exchanged across several frowarding devices $Fw(n)$ with $d(n)$ diameter between the supplicant and the AS. As the SDN network grows, the longest path will be equal to $d(n)$ which means the communication complexity of our proposed network design is $\Omega(Fw(n)*d(n))$. A resilient network architecture need to be designed adequately to meet the requierements of security, scalability and quality of service.

3.3. Scalability and performance: experimental analysis

For our experimental analysis, we have set up two labs: Lab1 (Figure 3) consists of a POX controller, 2 HP OF-switches, Microsoft Network Policy Server (NPS) which is used and configured to perform RADIUS authentication (Windows server 2008 R2 Enterprise Edition), DHCP Server, DNS server, Domain controller and Active directory Certificate Services. Two hosts are connected to OF-switch 1. OF-switch 2 connects POX controller to OF-switch 1. The DHCP and the RADIUS servers are both connected to OF-switch 2.

For Lab2, (Figure 4) instead of the HP OF-switch 1, we inserted a legacy HP switch with IEEE 802.1X support. Both the DHCP and the RADIUS servers are plugged into this switch. The POX controller is running over a Linux machine (OS: Debian; RAM: 4G; CPU: Intel core 2 Quad).

In both labs, the OF-switches are configured in proactive mode. The POX controller uses hostapd as an authenticator to forward the authentication requests.

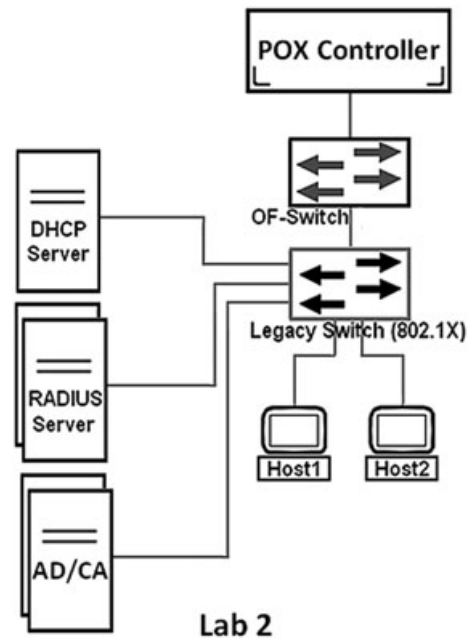


Figure 4. Lab 2

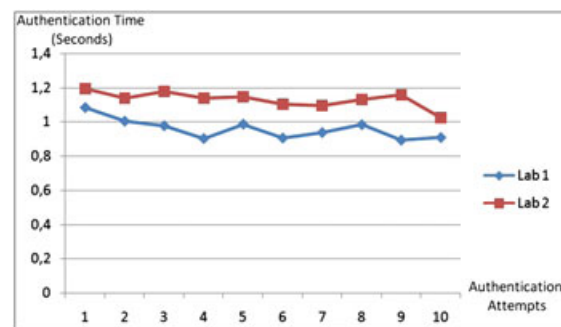


Figure 5. IEEE 802.1X authentication delays for Lab 1/Lab 2 in scenario 1.

We have created certificates with Active directory Certificate Services, defined Network policies and RADIUS attributes for NPS server. The adopted authentication method, in our scenario, is PEAP-EAP-MSCHAPv2. PEAP creates a secure tunnel between the supplicant and the NPS server based on the certificate. EAP-MSCHAPv2 authentication method relies on credentials to authenticate the user. The adopted authentication mechanism implements dynamic VLANs to allow or deny user access to the network from different locations.

DHCP server is configured with a scope for VLANs. We have defined three VLANs: VLAN 10, VLAN 20 and VLAN guest. Members of a given group will be placed in the appropriate VLAN as soon as authentication is successful. Otherwise, users are placed in VLAN guest.

Active Directory Domain Services is used to create the appropriate users and groups which can be used to place a given member into a corresponding VLAN. Concern-

ing the RADIUS servers, the configuration is synchronized from the master to the slave. With powershell we exported the configuration for the slave server, which contains a shared secret. If the master RADIUS server fails the next RADIUS server is invoked. Tehnically, slight delays in synchronization between the two RADIUS servers can be tolerated and will not affect the total authentication time.

We have conducted performance tests in two scenarios to deduce the authentication delays. In the first scenario, only the authentication flow is exchanged within (Figure 5) the network in both labs. In the second one, we have flooded the network with Ping flooding flow, and (Figure 6) in the meantime, we have tried to inject the authentication flow in order to observe the network responsiveness. We have used Wireshark tool in order to capture the authentication traffic and analyze the authentication delays.

The two hosts have been put through several successful authentications and re-authentications to extract the experienced authentication delays.

Multiple factors can influence the authentication delays among which there are switch processing speed, link speed

between different network elements, processing speed of network cards, distance, RAM, CPU, growing workload.

In scenario 1, the average authentication delay is nearly 0,9568 s for Lab 1 and 1,1205 s for Lab 2. In Lab 2, the average authentication process experiences more few milliseconds than that of Lab 1 regarding the delay, because the supplicant contacts a distant SDN controller (POX) and takes longer path to reach the servers: the supplicant sends an IEEE 802.1X message to OF-Switch 1, and then, the request is forwarded to OF-Switch 2. Next, POX controller (Authenticator) receives the message and contacts the NPS server through OF-Switch 2. Afterwards, a response is sent back to POX controller via OF-Switch 2. Finally, the response message travels across the two OF-switches to reach the supplicant. However, in Lab 1, the supplicant contacts the NPS server only through the legacy switch which reduces the time of the authentication process.

In scenario 2, the average authentication delay is 1,9712 s for Lab 1 and 2,3467 s for Lab 2.

As the network bandwidth is occupied by a growing network flow (Ping Flooding) and the network elements are under workload (Responding to ping messages), the authentication delays for scenario 2 are higher than those of scenario 1. We can notice a margin of 1,2262 s concerning Lab 2 when comparing the two scenarios and 1,0144 s for Lab 1.

We have also conducted another test under both labs to show the impact of the number of requests in the SDN on the authentication delay. We can notice in Figure 7 the experienced latency upon sending an increasing simultaneous authentication requests. The Authentication attempts have been reproduced three times and the average time value has been calculated for every flow of requests (i.e. 1, 30, 60...180).

The SDN controller could experience longer time when under workload before giving an authentication response. Besides, the IEEE 802.1X functionality is not

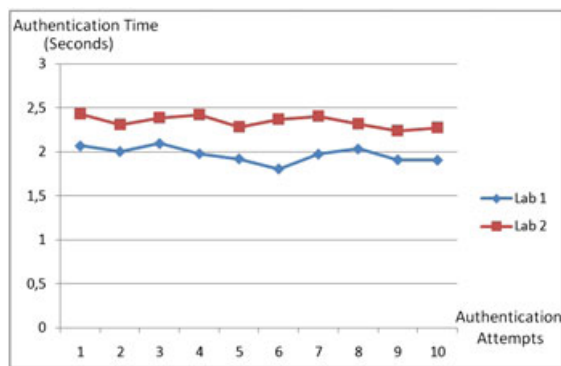


Figure 6. IEEE 802.1X Authentication delays for Lab 1/ Lab 2 in scenario 2.

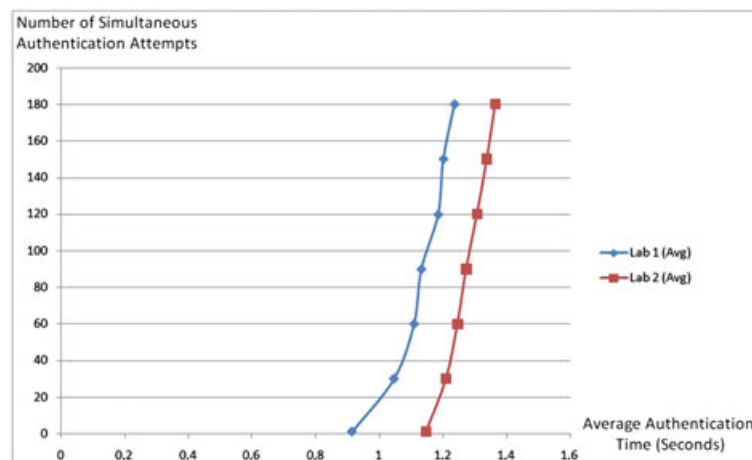


Figure 7. Impact of the number of requests in the SDN on the authentication delay.

usually suitable as a feature to be supported in SDN controller. Therefore, devolving some capabilities to the data plane, could sometimes significantly help reducing network latencies, improve the overall network performance and avoid network attacks such as Denial of Service attacks.

4. CONCLUSION

We have proposed a new network design which integrates IEEE 802.1X port based authentication in a new fashion for SDN. We have proved that the inclusion of IEEE 802.1X capabilities in SDN controller can be a serious challenge when it comes to the scalability of the SDN controller which is the brain of the whole architecture. Also, placing the authenticator inside the network has security and performance drawbacks. Our proposal can solve the scalability and security issues by directly verifying users identities and devolving the authenticator capabilities to the legacy switch. The insertion of the IEEE 802.1X legacy switch enhances the network control as it operates at the data link layer and does not require escalating to network or application layers for identification and verification. Moreover, the authentication flow has not to take longer path to reach the network components involved in the identification/verification of identities.

REFERENCES

- Kreutz D, Ramos FM, Esteves Verissimo P, Esteve Rothenberg C, Azodolmolky S, Uhlig S. Software-defined networking: a comprehensive survey. *Proceedings of the IEEE* 2015; **103**(1): 14–76.
- McKeown N, Anderson T, Balakrishnan H, Parulkar G, Peterson L, Rexford J, Shenker S, Turner J. OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review* 2008; **38**(2): 69–74.
- Scott-Hayward S, Natarajan S, Sezer S. A survey of security in software defined networks. *IEEE Communications Surveys & Tutorials* 2015; **18**(1): 623–654.
- Benton K, Camp LJ, Small C. OpenFlow Vulnerability Assessment. In *Proceedings of the 2nd ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking*. ACM: Hong Kong (the chinese university of Hong Kong), 2013; 151–152.
- IEEE 802.1X. Available at <https://tools.ietf.org/html/rfc3580> [Last accessed : October].
- Yeganeh SH, Tootoonchian A, Ganjali Y. On scalability of software-defined networking. *IEEE Communications Magazine* 2013; **51**(2): 136–141.
- van Asten BJ, van Adrichem NL, Kuipers FA. Scalability and resilience of software-defined networking: an overview 2014, arXiv preprint arXiv:1408.6760.
- Matias J, Garay J, Mendiola A, Toledo N, Jacob E. FlowNAC: Flow-based Network Access Control. In *Third European Workshop on Software-Defined Networks*, 2014.
- Mattos DMF, Henrique L, Toledo N, Jacob E. AuthFlow: authentication and access control mechanism for software defined networking. *Annals of Telecommunications* 2014; 1–9.
- Yamasaki Y, Miyamoto Y, Yamato J, et al. Flexible Access Management System for Campus VLAN Based on OpenFlow. In *International Symposium on Applications and the Internet, IEEE/IPSJ*, 2011.
- Kinoshita S, Watanabe T, Yamato J, et al. Implementation and Evaluation of an OpenFlow-based Access Control System for Wireless LAN Roaming. In *the 36th International Conference on Computer Software and Applications Workshops*, IEEE, 2012.
- RADIUS. Available at <https://tools.ietf.org/html/rfc3579> [Last Accessed: October 2015].
- Cai Z. Maestro: Achieving scalability and coordination in centralized network control plane. *Ph.D. Dissertation*, Rice Univ., Houston, TX, USA, 2011.
- Tootoonchian A, Ganjali Y. HyperFlow: A distributed control plane for OpenFlow. In *Proceedings of the Internet Network Management Conference on Research*, 2010.
- Yu M, Rexford J, Freedman JM, Wang J. Scalable flow-based networking with difane. *SIGCOMM Computer Communication Review*, ACM 2010; **41**(4): 351–362.
- Dixit A, Hao F, Mukherjee S, Lakshman T, Kompella R. Towards an elastic distributed SDN controller. In *Proceedings of the 2nd ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking*, ser. HotSDN 13. ACM: New York, NY, USA, 2013; 7–12.
- Dixit A, Hao F, Mukherjee S, Lakshman TV, Kompella R. ElastiCon: an elastic distributed sdn controller. In *Proceedings of the 10th ACM/IEEE Symposium on Architectures for Networking and Communications Systems*. ACM: Santa Clara, CA, USA, 2014; 17–28.
- Curtis AR, Mogul JC, Tourrilhes J, Yalagandula P, Sharma P, Banerjee S. DevoFlow: scaling flow management for high performance networks. *Computer Communication Review*, ACM 2011; **41** (4): 254–265.
- Mogul JC, Congdon P. Hey, you darned counters!: get off my asic! In *Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks*, ser. HotSDN 12. ACM: New York, NY, USA, 2012; 25–30.
- Koponen T, Casado M, Gude N, Stribling J, Poutievski L, Zhu M, Ramanathan R, Iwata Y, Inoue H, Hama T, Shenker S. Onix: a distributed control platform for large-scale production networks. In

- Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI10. USENIX Association: Berkeley, CA, USA, 2010; 1–6.
21. Hassas Yeganeh S, Ganjali Y. Kandoo: A framework for efficient and scalable offloading of control applications. In *Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks*, ser. HotSDN 12. ACM: New York, NY, USA, 2012; 19–24.
 22. Maestro Platform. Available at <http://code.google.com/p/maestro-platform/> [Last accessed : November 2015].
 23. Tootoonchian A, Gorbunov S, Ganjali Y, Casado M, Sherwood R. On controller performance in software-defined networks. In *Proceedings of the 2nd USENIX Conference*. Hot-ICE Netw. Serv. USENIX Association: , 2012; 10.
 24. Jarschel M, Oechsner S, Schlosser D, Pries R, Goll R, TranGia P. Modeling and performance evaluation of an OpenFlow architecture. In *2011 23rd International Teletraffic Congress (ITC)*, Sept 2011; 1–7.